ADS Assignment Report - Ishan Batra - CSE 3 - 240485

This report is only a summary of what was done and what was found. All the code and detailed results are at: https://github.com/IshuIsAwake/advanced_data_science

Part 1: Avazu Click Rate Prediction: https://www.kaggle.com/c/avazu-ctr-prediction

1. Business Understanding

An ad platform shows ads on websites and inside apps. We assume it gets paid when an ad is clicked, so it wants to know which ads are likely to be clicked and put its budget there.

What we predict: the column *click*, which is 1 if an ad was clicked and 0 if not. The share of ads that get clicked is the **click-through rate (CTR)**.

Questions we asked: What share of ads get clicked? Which devices, apps, websites, positions and hours get more clicks? Can a model predict clicks better than a simple guess? Where should the budget go?

2. Dataset Summary and EDA

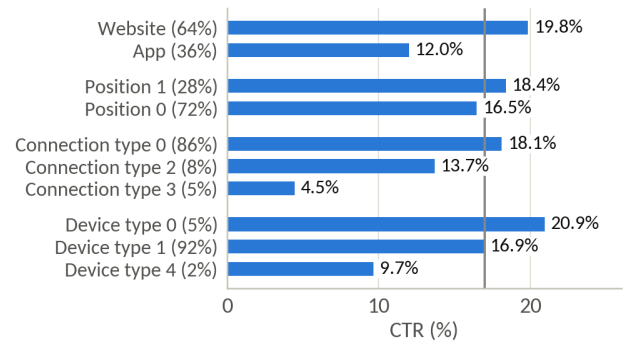
The data: click logs from Avazu's Kaggle competition: 40.4 million ads shown over 10 days in October 2014, already split into training (80%), validation (10%) and test (10%) rows. The dfSummary overview is in the notebook *01_eda*.

Our sample: every 40th row of each split, about 1 million rows, because that is what fits in our graphics card's memory. All 40 million rows come later, with batching (training in small chunks) and a neural network written from scratch. The sample still looks like the full data: the quietest hour has 11,877 rows in full and 298 in the sample, the busiest has 358,731 and 8,967, and hourly CTRs differ only as much as chance would.

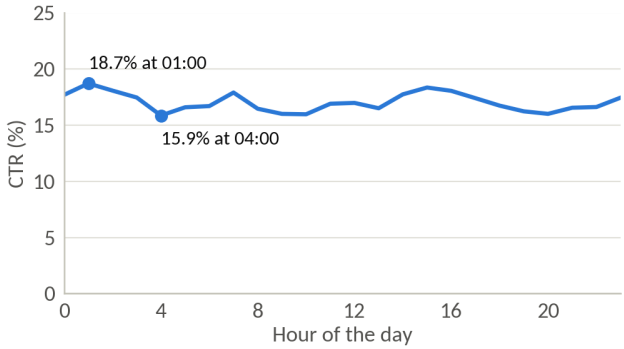
Columns	What they hold	Why they matter
click	Clicked or not (1 or 0)	This is what we predict
hour	Date and hour, stored as a number	People may click more at some times
banner_pos	Position of the ad, as a code	Some positions get more attention
site_id, site_domain, site_category	Which website, as scrambled text codes	Some websites get far more clicks
app_id, app_domain, app_category	Which app, as scrambled text codes	Some apps get far more clicks
device_id, device_ip, device_model, device_type, device_conn_type	Which device, its model and type, and its connection, as codes	They go along with clicks. The ID and IP point to individual people
C1, C14 to C21	Hidden columns, their meaning is not disclosed	They still help predict clicks

- **Size:** 24 columns, all of them categories, even the ones that look like numbers.
- **Missing values:** no blank cells, but some values stand in for "unknown". One device ID sits on 82.5% of rows, with 377,512 different IP addresses, which no single device could have. App ads carry filler values in the website columns and the reverse, so we read website categories on website ads only and app categories on app ads only. This is our reading of the patterns, since the data has no definitions.
- **Clicks and no clicks:** 17% clicked and 83% did not. A model that always says "no click" would be 83% correct and still useless, so we do not judge models by accuracy.

Some placements get far more clicks
In brackets: share of all ads. Grey line: all ads, 17.0%



The hour barely changes the CTR
It stays between 16% and 19% all day



- Website ads get a higher CTR than app ads: 19.8% against 12.0%.
- Position 1 gets 18.4%, position 0 16.5%. Connection matters more: type 0 gets 18.1%, types 3 and 5 under 5%.
- Device type 0 gets 20.9%, type 1 (92% of ads) 16.9%, types 4 and 5 about 9.5%. These codes are not defined.

- Within websites, the 8 largest categories run from 28.3% to 3.3%. Within apps, they run from 25.0% to 9.0%.
- The hour barely matters for CTR, even though traffic varies from 2% to 6% of ads from hour to hour.

3. Machine Learning

Preparing the data:

- Split *hour* into hour of the day and day of the week.
- Removed the row number, and the device ID and IP address, because they identify individual people.
- Removed columns that add little, and measured what each one cost in log loss (explained below, lower is better). Removing C14 saved 421 columns and raised log loss by 0.0009. Removing C18, C19 and C21 saved 108 columns and raised it by 0.0001. Removing app_domain saved 24 and raised it by 0.0001. Removing C1 saved 5 and barely changed it. With all of them kept, the model does not fit in memory.
- Grouped values under 0.025% of training rows into "other", and turned categories into 1,356 yes or no columns.
- The test rows played no part in any of this, so nothing leaked from them into the model.

Model	In simple words	Log loss	ROC-AUC	PR-AUC	F1
Baseline	Gives every ad the same predicted CTR, explained below.	0.458	0.500	0.171	0
Logistic regression	Gives each column a weight, adds them up and turns the total into a predicted CTR.	**0.405**	**0.734**	**0.358**	**0.408**
Weighted logistic regression	The same, but clicks count about 5 times as much in training, so the rare clicks are not ignored.	0.603	0.734	0.357	0.407
Weighted linear SVM	Looks for a line that separates clicks from no clicks. It gives a score, not a predicted CTR, so it has no log loss.	-	0.717	0.312	0.396

- **Log loss:** how close the predicted CTRs were to what happened. Lower is better.
- **ROC-AUC:** take one clicked ad and one unclicked ad. This is how often the model gives the clicked one the higher score. 0.5 is guessing and 1 is perfect.
- **PR-AUC:** how many of the ads with the highest scores were really clicked. Guessing scores about 0.17.
- **F1:** for a yes or no call, the balance between catching clicks and avoiding false alarms. We set the cut-off on the validation rows by best F1, because the cut-off with the best accuracy, near 0.5, catches only 7% of clicks.

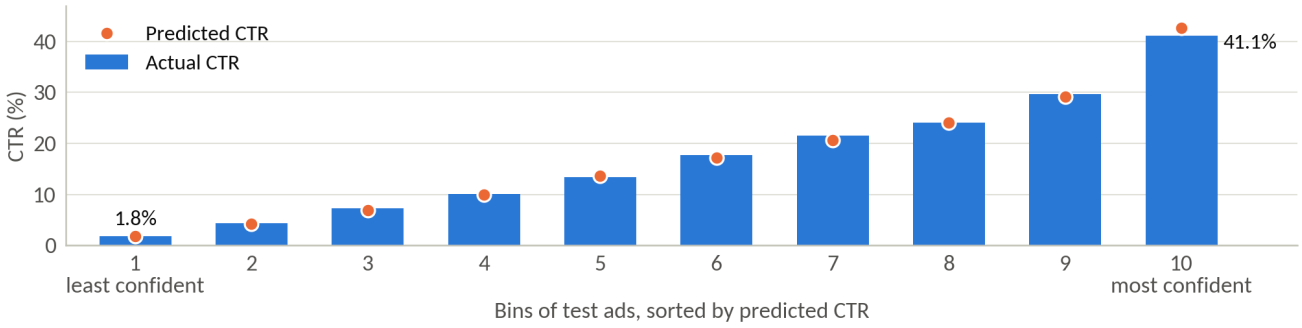
Scores are on 101,073 test rows. **Plain logistic regression is the best model:** its log loss is 11.5% lower than the baseline and it ranks ads better than the SVM. We wrote the models from scratch in PyTorch. Logistic regression matches the standard library (sklearn) to 3 decimals, and the SVM comes close (ROC-AUC 0.717 against 0.713).

The baseline gives every ad the average CTR of the training rows, 17%, and uses nothing about the ad. We picked it because it is the simplest possible model, and no other fixed number gets a lower log loss on the training rows, so beating it means a model learned from the columns. It cannot rank ads (ROC-AUC 0.5) and never says "click" (F1 0).

4. Inference and Interpretation

How this graph is made: we took the raw output of logistic regression, its predicted CTR, for every ad in the test set, sorted the ads by it and split them into 10 equal bins. Bin 1 holds the ads where the model is least confident of a click, and bin 10 the ads where it is most confident. For each bin we compare the average predicted CTR with the actual CTR.

Where the model predicts a higher CTR, the actual CTR is higher



- **Ranking:** where the model is less confident, the actual CTR is lower, and where it is more confident, the actual CTR is higher. Bin 1 gets 1.8% and bin 10 gets 41.1%, a 23 times gap.
- **Predicted against actual:** the two are close in every bin. In bin 10 the model predicts 42.7% and the actual CTR is 41.1%. The other nine bins are within about 1 point.

- **Overfitting:** log loss is 0.402 on the training rows and 0.405 on the test rows, so the model is not just memorising the training rows.
- **Weighted model:** it ranks ads just as well (ROC-AUC 0.734), but its predicted CTRs are far too high: 44% on average against an actual 17%.
- **Most useful columns:** app, C17 (a hidden column), device model and website raise log loss the most when removed.
- **Prediction is not cause.** The model shows what goes along with clicks. Position 1 gets a higher CTR, but better ads may simply be given position 1. Only an experiment that places ads at random (an A/B test) can show whether moving an ad changes its CTR.
- **Limits:** one row in 40 used; rare values grouped; hidden and scrambled columns that are hard to explain; only 10 days, with test rows from the same days, so new days will probably score a bit worse; a simple model that misses combinations (this app on this device); no history per user; no prices.

5. Recommendations

1. **Rank ads by predicted CTR × price per click.** Run each ad through plain logistic regression to get a predicted CTR, multiply it by the price per click (what the client pays us for the ad), then sort. The ads with the highest predicted CTR × price per click should get the most budget. The price is not in our data, so it has to come from the platform.
2. **Put the budget on the highest predicted CTRs.** The 30% of ads with the highest predicted CTR got an actual CTR of about 32%, and the lowest 30% about 4.5%.
3. **Test before shifting budget.** Website ads, connection type 0 and device type 0 have higher CTRs than the rest. Connection types 3 and 5, device types 4 and 5, and the weakest website categories (f66779e6 and 76b2941d, under 4%) are the first places to cut. These are patterns and may not be causes, so try them on a small share of the budget in an A/B test.
4. **Do not use the weighted model's predicted CTR,** because it is far too high.
5. **Next steps:** train on all 40 million rows, add a neural network that can learn combinations, add history per user, and check the model on the last day of data.

Part 2: Evaluating a Job Training Programme

Important: this data is synthetic, made for practice. It is not evidence about any real Indian government programme.

Business Understanding

The government ran a lottery and offered subsidised job training to the winners. Those offered could enrol or decline, and a few not offered enrolled anyway. Employment and earnings were recorded at 18 and 30 months. The government asks: what does offering training do, what does the training itself do, and do OBC, SC and ST applicants benefit? We used the linked file: 40,000 applicants in 45 districts (the assignment says 10,000 in 16).

1. Data Understanding and Randomisation

- **The data:** 40,000 applicants and 25 columns. The only blanks are in training type, exactly for the 23,140 people who did not enrol, so they mean "no training".
- **Was the lottery fair?** If it was, those offered and not offered should differ only by chance. We compared them on 14 background facts (past earnings, schooling, age, caste and so on). The largest gap is the SC share (18.3% against 17.4%, or 0.025 standard deviations), and it is also the only p value below 0.05 (p = 0.017). At a 5% false positive rate, 14 tests are expected to give 14 × 0.05 = 0.7 false positives, so one is not surprising. With the Bonferroni correction the cut-off becomes 0.05 / 14 = 0.00357, and 0.017 is clearly above it. In every state, exactly 2 in 3 applicants were offered training. The lottery looks fair.

	Did not enrol	Enrolled	Total	Enrolment rate
Not offered	12,927	406	13,333	3.0%
Offered	10,213	16,454	26,667	61.7%

- **First stage:** the offer raised enrolment by 58.7 points (95% confidence interval 58.0 to 59.3).
- **Noncompliance:** 38% of those offered never enrolled, and 406 people enrolled without an offer. So being offered and being trained are not the same. Comparing people who enrolled with people who did not would mix the effect of training with the kind of person who chooses to enrol.

2. Intent-to-Treat (ITT) Analysis

Every number in this section compares everyone offered training with everyone not offered. Someone offered counts as offered even if they never enrolled, and someone not offered counts as not offered even if they enrolled anyway. The lottery made the two groups alike, so the difference is caused by the offer. These numbers are the effect of being offered training. The effect on the people who actually took it is section 3.

Outcome	Offered	Not offered	Difference in means	95% confidence interval
Employed at 18 months	46.2%	43.1%	+3.1 points	2.1 to 4.1
Employed at 30 months	46.9%	43.0%	+4.0 points	2.9 to 5.0
Earnings at 18 months	₹58,712	₹52,161	+₹6,551	₹5,036 to ₹8,067
Earnings at 30 months	₹76,406	₹67,329	+₹9,076	₹7,084 to ₹11,069

- **How we test it:** we want to see whether zero, meaning no effect, lies outside the 95% confidence interval (CI) of the difference in means. The CI is the difference ± 1.96 standard errors, from Welch's t-test, which does not assume both groups vary equally (its standard errors are heteroskedasticity-robust). The standard error says how much the difference would move by chance from one sample to the next. 1.96 is the 95% value of the t distribution, which is effectively normal with this many people. If we repeated the study 100 times, about 95 of the CIs would contain the true effect.
- **Result:** zero lies far outside all four CIs (every p is below 0.001), so the offer raises both employment and earnings.
- **For policymakers:** on average, an applicant offered training earns about ₹9,000 more at 30 months, 13.5% above those not offered, and employment is 4.0 points higher. This counts the 38% who never took the training.
- **Does the effect grow over time?** The earnings effect rises by ₹2,525 from 18 to 30 months (95% CI ₹212 to ₹4,838), though as a share of the not offered group's earnings it barely moves (12.6% to 13.5%). The employment effect rises by 0.84 points, which could be chance (p = 0.24).
- **Not done yet:** repeating this with baseline covariates (each person's background), as the assignment asks.

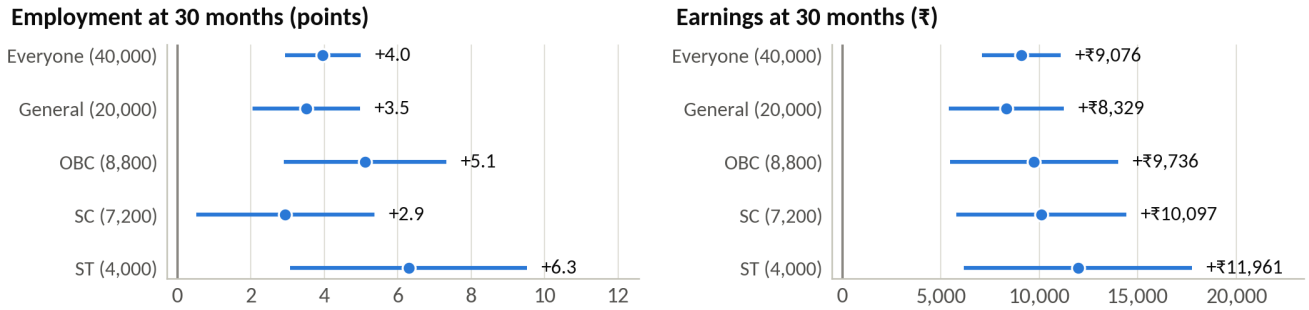
3. Instrumental Variables and LATE

Not done yet. This step would estimate the effect of the training itself on the people who actually took it, using the Wald estimator and two-stage least squares, and cover monotonicity, the exclusion restriction, compliers, always-takers and never-takers. Until then the government's second question stays open.

4. Earnings, Employment and Heterogeneous Effects

Zero earnings: at 30 months, 57.0% of those not offered and 53.1% of those offered earn nothing, and earnings are 0 exactly when someone is not employed. So the ₹9,076 has two parts: ₹6,201 from more people working and ₹2,875 from offered workers earning more (arithmetic only). Comparing the employed alone is not fair, because the offer changes who works: offered workers have 0.11 fewer years of schooling (p = 0.02), while the full groups match.

Heterogeneous effects means the effect may differ between kinds of people. We ran the same comparison inside each social category, which is fair because the lottery ran in every group and a category is fixed before the offer.



Each dot is the difference in means and each line is its 95% CI. The number of people is in brackets.

Group	Employed, 18 months	Employed, 30 months	Earnings, 18 months	Earnings, 30 months
General	+3.6	+3.5	+₹6,687	+₹8,329
OBC	+2.4 *	+5.1	+₹6,871	+₹9,736
SC	+4.0	+2.9 *	+₹8,229	+₹10,097
ST	+1.5 *	+6.3	+₹4,091 *	+₹11,961

Employment is in percentage points. * does not pass the stricter Bonferroni test explained below.

- **Does each group gain?** Yes. At 30 months every CI is above zero. ST has the largest differences and the widest CIs, as the smallest group.
- **Do groups gain different amounts?** We cannot say. We tested all 24 gaps between groups directly and none has p below 0.05 (smallest p = 0.10). ST's lead over General on 30-month earnings is ₹3,632, with a 95% CI from −₹2,849 to ₹10,114.
- **Do ST's gains come later?** Possibly. Its 18-month effects are not clear, and they rise by 4.8 points and ₹7,870 by 30 months, but neither rise passes the Bonferroni test for these 8 comparisons.
- **The risk of many tests:** the table holds 16 tests. If there were no real effects and the tests were independent, the chance of at least one false positive would be 56%. So we also applied the Bonferroni test, which judges each result at 0.05 / 16. 12 of the 16 still pass, including all four 30-month earnings effects.

5. Inference and Recommendations

Question	Answer	Why
Continue the programme?	Yes	The offer clearly raises employment and earnings.
Expand it?	Cannot say yet	The gain of ₹9,076 is clearly above zero (statistically significant) and 13.5% of the not offered group's earnings (a meaningful size). Whether it is worth paying for (policy significance) depends on the cost per offer, which is not in the data.
Redesign it?	Look at take-up	38% of offers go unused, and each one dilutes the effect per offer. Find out why people decline.
Target one group?	No	Every group gains, and nothing shows that any group gains more.

- **Assumptions:** the offer was random (the checks in section 1 support this), one applicant's offer does not change another's outcome, and outcomes were measured the same way for both groups.
- **Limits:** synthetic data; no costs; the effect of the training itself and the ITT with covariates not done; earnings period not defined; no adjustment for the lottery being run within states.
- **Future evaluations:** record the cost per offer, follow people for longer, and use larger samples for small groups.